

多言語対応 VITSベース TTS システムの構築ガイド

環境構築と依存関係インストール

1. **Python仮想環境の準備:** Windows上でPythonの仮想環境（例: `uv` という名前）を作成・有効化します。コマンド例（PowerShellの場合）：

```
python -m venv uv # 仮想環境フォルダ作成
.\uv\Scripts\Activate.ps1 # 仮想環境をアクティベート
```

その後、Pythonやpipのパスが仮想環境内のものになっていることを確認します。

2. **必要パッケージのインストール:** 仮想環境内で以下の主要ライブラリをインストールします。
3. PyTorch（CUDA対応版を使用する場合は適切なCUDAツールキットに対応したものをインストール）
4. Hugging Face Transformers ライブラリ（`transformers`）
5. XPhoneBERTに必要なテキスト前処理ライブラリ `text2phonemesequene` ¹
6. BigVGANボコーダ用のライブラリ（PyPI経由で提供されています: `pip install bigvgan`）
7. その他ユーティリティ: NumPy、Matplotlib、SoundFile/Librosa など（音声ファイル保存やスペクトログラム表示に使用）

コマンド例:

```
pip install transformers text2phonemesequene bigvgan matplotlib soundfile
```

補足: 上記ではCPU版PyTorchが入る可能性があります。GPUを利用する場合は、PyTorch公式サイトの指示に従い適切なコマンドでインストールしてください（例: `pip install torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cu118` など）。

XPhoneBERTの導入と音素変換

1. **XPhoneBERTモデルとトークナイザの読み込み:** Hugging Faceの事前学習済みモデル `vinai/xphonebert-base` を利用します。 `transformers` から `AutoModel` および `AutoTokenizer` を使用してモデル本体とトークナイザをロードします ²。さらに、XPhoneBERT用に提供されている `Text2PhonemeSequence` クラスを用いてテキストを音素列に変換します ³。

```
import torch
from transformers import AutoModel, AutoTokenizer
from text2phonemesequene import Text2PhonemeSequence

# XPhoneBERTモデルとトークナイザのロード
```

```
xphonebert = AutoModel.from_pretrained("vinai/xphonebert-base")
tokenizer = AutoTokenizer.from_pretrained("vinai/xphonebert-base")
```

1. **対応言語の音素エンコーダ設定:** `Text2PhonemeSequence` を使用して、各言語それぞれに対応する音素変換器を準備します。`Text2PhonemeSequence` の `Language` 引数にはISO 639-3コードを指定します⁴。例えば、日本語は `'jpn'`、英語（米国）は `'eng-us'`、中国語（北京官話）は `'cmn'` を指定します。

各言語用の音素変換器を初期化（CUDA対応の場合 is_cuda=True）

```
phonemizer_ja = Text2PhonemeSequence(language='jpn', is_cuda=True) # 日本語
phonemizer_en = Text2PhonemeSequence(language='eng-us', is_cuda=True) # 英語（米語）
phonemizer_zh = Text2PhonemeSequence(language='cmn', is_cuda=True) # 中国語（官話）
```

- **注意:** CJK言語（中日韓など）では文章中に空白で区切られた単語境界がありません。そのため、`Text2PhonemeSequence` に渡す前に**単語分かち書き**を行う必要があります⁵⁶。例えば日本語では形態素解析（MeCabやspaCyのjaモデル等）、中国語ではjieba等で単語ごとに空白を挿入してください。英語についても、必要に応じて正規化（例： 全て小文字にする、数字を単語に展開する等）を行ってから音素変換すると良いでしょう⁶。

- **テキストから音素列への変換:** サンプルテキストを各言語で用意し、上記のphonemizerで音素列に変換します。入力文字列は**単語単位で空白区切りされた状態**で渡します⁷⁸。例えば:

- 日本語: `"これはテストテキストです。"` → `"これ は 、 テスト テキスト です ."`（単語毎・記号毎に空白を挿入）
- 英語: `"This is a test."` → `"this is a test ."`（必要ならば前処理で小文字化と句読点前後に空白）
- 中国語: `"这是一个测试。"` → `"这 是 一 个 测 试 ."`（簡易には各漢字・単語の間に空白を入れる）

変換コード例（日本語のケース）:

```
sentence_ja = "これは、テスト テキスト です。" # すでに分かち書き・正規化済みの文字列
phoneme_seq = phonemizer_ja.infer_sentence(sentence_ja)
print(phoneme_seq)
# 例: phoneme_seq が "korewa,tesutotekisutodesu." のような音素系列になる
```

上記の `phoneme_seq` には、スペースで区切られた音素記号列が文字列として得られます。次に、この音素列をXPhoneBERTのトークナイザでID列に変換します⁸。

```
inputs = tokenizer(phoneme_seq, return_tensors="pt")
```

1. **音素ベクトル埋め込みの取得:** 準備した `inputs` をXPhoneBERTモデルに入力し、**隠れ状態特徴量**を取得します⁹。通常、`AutoModel` でロードしたモデルの出力 `features` には `last_hidden_state`（最終層の隠れベクトル列）が含まれます。これが各音素に対応した多言語音素埋め込みベクトルとなります。コード例:

```
with torch.no_grad():
    features = xphonebert(**inputs)
    phoneme_embeddings = features.last_hidden_state # サイズ: [バッチサイズ, 音素トークン数, 埋め込み次元]
```

ここで得られる `phoneme_embeddings` は、各音素トークン（系列長に相当）について768次元（BERT-baseと同じ次元）の埋め込みベクトルを含むテンソルです。このベクトル列を後段のVITSモデルに渡すことで、テキストエンコーダ部分を強化することが狙いです ¹⁰ ¹¹。

メモ: この段階ではXPhoneBERTによる多言語対応の音素表現が得られました。後続のVITSモデルにこの表現を統合することで、少ない学習データでも音質やプロソディが向上する可能性が示されています ¹⁰。

事前学習済みVITSモデルの選定と読み込み

- 適切なVITSモデルの選定:** 次に、XPhoneBERTの出力と統合するTTSモデルとしてVITSアーキテクチャの事前学習モデルを用意します。多言語（日本語・英語・中国語）対応の単一モデルが理想ですが、公開されているものが無い場合は**言語ごとのモデル**を用意して切り替える方法もあります。代表的な選択肢:
- Facebook MMS-TTSモデル:** Meta社の「Massively Multilingual Speech」プロジェクトのTTSモデルです ¹²。1100以上の言語に対応し、各言語ごとにVITSベースの生成器モデルが公開されています ¹³。例: 英語モデル `facebook/mms-tts-eng`、日本語モデル `facebook/mms-tts-jpn` など。中国語(普通话)についてはMMS-TTSでの公開はありません（2023年時点） ¹⁴ ¹⁵。
- 学術研究版マルチリンガルVITS:** Edressonらの“YourTTS” ¹⁶ はVITSを拡張して英語・ポルトガル語・フランス語等で学習されたモデルです。ただし日本語・中国語は含まれていません。
- 各言語の単独モデル:** 英語であればLJSpeechデータで学習した単一話者モデル、もしくはLibriTTSやVCTKで学習した多話者モデルが入手可能です ¹⁷。中国語はAISHELL-3多話者コーパスで学習したVITSモデル ¹⁸ やBZNSYP単一話者モデル等が存在します。日本語はJSUTやJVSデータセットで学習されたVITSモデル（例: ESPnetによる日本語VITS ¹⁹ など）が利用できます。例えば、Sherpaプロジェクトでは中国語+英語の2言語に対応したシングルスピーカーモデル「vits-melo-tts-zh_en」が公開されています ²⁰。

初期検証としては、各言語で安定した発話が得られるモデルを別々に用意し、パイプラインの動作確認をする方法が確実です。ここでは例として:

- 日本語: 日本語単一話者のVITSモデル
- 英語: LJSpeechで訓練済みのVITSモデル
- 中国語: AISHELL-3ベースの多話者VITSモデル

を使用するものと仮定します（実際には手元にある事前学習モデルに置き換えてください）。

- モデルの読み込み:** VITSの実装および事前学習済み重みのロードには、モデルの提供元に応じた手順を踏みます。2通りのアプローチがあります。
- (A) Hugging Face Transformers経由:** 上記MMS-TTSのようにHugging Face Hubで提供されている場合、`transformers`の `VitsForConditionalGeneration`（または `AutoModel`）でロード可能です。例えば英語モデルなら:

```
from transformers import VitsForConditionalGeneration, AutoTokenizer
vits_model_en = VitsForConditionalGeneration.from_pretrained("facebook/mms-tts-eng")
tokenizer_en = AutoTokenizer.from_pretrained("facebook/mms-tts-eng")
```

このようにモデルとトークナイザをロードし、テキストを入力すると音声波形を生成できます。**注意:** MMS-TTSの場合、モデル内部で自動的にテキスト→音素変換（phonemize）が行われる仕様があります²¹。本ガイドではXPhoneBERTで音素に変換済みのため、トークナイザ側でphonemize機能をオフにする必要があります（VitsTokenizerのphonemize=False設定など）。

3. **(B) オリジナル実装を使用:** 学習元のGitHubレポジトリや独自チェックポイントがある場合、そのコードベースをクローンしてモデル構造を定義し、提供されたチェックポイントファイル（例: G_****.pth）をロードします²²。例えば公式VITS実装やCoqui TTSフレームワークを使う場合、対応するconfig.jsonを読み込みネットワークを構築し、重みをロードします。コード例（疑似コード）:

```
import torch
from VITS_impl import SynthesizerTrn # VITSモデルクラス（実装に応じて異なる）
# 設定ファイルの読み込みとモデル初期化
config = load_config("config.json")
vits = SynthesizerTrn(**config["model"])
# 重みロード
ckpt = torch.load("G_checkpoint.pth", map_location="cpu")
vits.load_state_dict(ckpt["generator"])
vits.eval().to(device)
```

その後、テキストを音素ID列に変換し（モデルに付属の辞書やphoneme setに合わせる必要があります）、モデルの推論関数を呼び出します。

4. **XPhoneBERT出力との統合:** ロードしたVITSモデルにXPhoneBERTの埋め込み出力を統合する方法はいくつか考えられます。
5. **テキストエンコーダ層の置換:** VITSのテキストエンコーダ部分（通常は音素IDをembeddingし畳み込みやTransformerで処理する部分）を省略し、代わりにXPhoneBERTから得たphoneme_embeddingsを直接モデルの残りに投入します¹¹。具体的には、phoneme_embeddings（サイズ[1, T_phoneme, 768]）をVITS内部の埋め込み次元に射影する全結合層を新設し、その出力を本来のテキストエンコーダ出力に相当する箇所に接続します。
6. **埋め込みの追加入力:** VITSのテキストID埋め込みに対し、XPhoneBERTの埋め込みを加算あるいは連結する方法もあります。例えば、まず通常通り音素IDをVITSのembedding層でベクトル化し、それにXPhoneBERT埋め込みを加算してからVITSデコーダに送る、といった実装も可能です。

これらの変更を行う場合、既存モデルとの不整合（埋め込み次元の差異など）が生じるため、小規模でも良いので統合後のモデルを調整・ファインチューニングすることが望ましいです。初期検証の段階では、大きな学習は行わずとも、上記変更によってモデルが動作するかどうかを確認します。その際、音質が低下したり発話がおかしくなる可能性があります。これは統合時に追加したレイヤーが最適化されていないためです。あくまで「動く」ことの確認が目的であれば、ここでは細かいチューニングは後回しにします。

参考情報: XPhoneBERTの公式実装では、テキスト→音素列変換に `text2phonemessequence` を使い、その音素列をVITS2モデルに入力するデモが公開されています²³。このように、まず音素ID列を取得してからXPhoneBERT組み込みのVITSで推論するという手順になります。

BigVGANボコーダーの導入と読み込み

1. **BigVGANモデルの準備:** BigVGANは高品質な汎用神経ボコーダーで、入力の上スペクトログラムから音声波形を生成します²⁴。NVIDIA提供の公式実装では、Hugging Face Hubに学習済みモデルが公開されており、そこで用意されたコードを使って簡単に推論が可能です²⁵²⁶。例えば22.05kHzサンプリング・80次元上スペクトログラム対応のモデルを使う場合、以下のように読み込みます (PyPI版をインストール済みの場合)²⁷²⁸:

```
import bigvgan
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# Hugging Face上のプリトレーニング済みBigVGANボコーダーをロード
vocoder = bigvgan.BigVGAN.from_pretrained("nvidia/bigvgan_22khz_80band",
use_cuda_kernel=False)
vocoder.remove_weight_norm() # 重み正規化を除去して推論を安定化
vocoder.eval().to(device)
```

上記では22kHz・80バンドモデル用のモデルを選択しています²⁸。もしVITS側の上スペクトログラムが24kHz・100バンドなど異なる次元の場合、対応するBigVGANモデル (例: `"nvidia/bigvgan_24khz_100band"` 等) を選んでください²⁹。BigVGANは大規模なデータで学習されており、複数言語の音声も含まれるため、**どの言語の上スペクトログラムに対しても高品質な波形再構成が期待できます**²⁴。

テストデータでの推論パイプライン

1. **入力テキストの準備:** 日本語・英語・中国語それぞれで短いテスト文を用意します (前述のとおり、事前に単語分割・正規化を済ませてください)。例えば:

- 日本語: `"これはテストです。"`
- 英語: `"This is a test."`
- 中国語: `"这是一个测试。"`

これらをそれぞれの `Text2PhonemeSequence` で音素系列に変換します (ステップ5参照)。そしてXPhoneBERTの `tokenizer` でテンソル化し、XPhoneBERTモデルで埋め込み特徴を得ます (ステップ6参照)。

2. **VITSによる上スペクトログラム生成:** 続いて、選定したVITSモデルを使って音声の中間表現である**上スペクトログラム**を生成します。ここでは各言語ごとに対応するVITSモデルを使う想定です。例えば日本語テキストには日本語VITSモデルを使用します。

- **XPhoneBERT統合なしで試す場合** (まず通常のパイプライン検証): 音素系列をそのままVITSモデルに与えて音声を合成します。Hugging Face経由のモデルであれば、テキストまたは音素列文字列をトークナイザでID化し、`model.generate()` や `model(**inputs)` で波形出力が得られます²¹。公式実装の場合は `vits.inference(text_ids, ...)` のような専用関数で上スペクトログラムや波形を計算します。例えばCoqui TTSのVITS実装では、`model.inference(text, speaker_id, language_id)` で直接波形生成まで可能です。ここでは

最終出力波形ではなく**中間のメルスペクトログラム**が欲しいため、モデル内の**デコーダ出力**を取得する必要があります。

- HuggingFace版VITSの場合、`VitsForConditionalGeneration` の `generate` メソッドは最終波形を返しますが、`output_hidden_states=True` などのオプションで隠れ層（メルスペクトログラム相当）の出力を得られる可能性があります³⁰³¹。ドキュメントによれば、VITSモデルの `hidden_states` にHiFi-GANに渡す前のスペクトログラムが含まれるとの記述があります³²。従って、`outputs = model.generate(**inputs, output_hidden_states=True)` とし、`outputs.hidden_states` からスペクトログラムを取り出すことが考えられます。
- 独自実装の場合、VITSのデコーダ出力であるメルスペクトログラム（もしくはそれに類する表現）を直接取り出します。例えば公式実装の `SynthesizerTrn` では、`model(x, ..., return_mel=True)` のような引数設定でメル出力を得ることもできます。もしそのようなAPIが無ければ、モデルのコードを読むか、一度波形まで生成してしまい**そこから改めてメルスペクトログラムを計算**する方法もあります。
- **XPhoneBERT統合後の実行**: XPhoneBERTの `phoneme_embeddings` を統合する改造を施した場合は、そちらのモデルを使って推論します。手順は概ね同じですが、入力として音素ID列ではなく前段で得た `phoneme_embeddings` を内部に渡す処理になっている点が異なります。適切に統合できていれば、あとは通常のVITS推論と同様にメルスペクトログラムが得られます。

いずれの場合も、**メルスペクトログラム（次元: [メルバンド数, フレーム数]**）が得られればOKです。例えば、得られた `mel` テンソルの形が `(1, 80, T)`（バッチ次元1、メルバンド80、フレーム長T）であることを確認します。

3. **BigVGANによる波形再構成**: 次に、ステップ12で得られたメルスペクトログラムをBigVGANボコードに入力して最終音声波形を生成します。これは数行のコードで可能です。例えば:

```
mel_tensor = torch.tensor(mel).unsqueeze(0).to(device) # メルをテンソルに（必要なら次元合わせ）
with torch.no_grad():
    audio_tensor = vocoder(mel_tensor) # 波形生成
audio = audio_tensor.squeeze().cpu().numpy() # 1次元の音声波形配列に変換
```

上記で `audio` が生成された音声データ（numpy配列）です²⁷。生成後、必要に応じて振幅を正規化したりクリッピングしたりします（通常はそのまま大丈夫ですが、場合によってはピークが大きすぎる場合があります）。

メモ: BigVGANの推論は高速ですが、CPU環境では時間がかかる可能性があります。短いテスト文であれば問題ありませんが、長い文章ではCUDA GPUの使用を検討してください。

4. **音声ファイルへの保存**: 得られた音声波形データを `.wav` ファイルとして保存します。標準的なPCM音声にするには、サンプリング周波数をVITS/BigVGANに合わせます（22kHzモデルなら22050 Hz等）²⁸。Pythonでは `soundfile` や `scipy.io.wavfile`、`librosa` などで保存可能です。例:

```
import soundfile as sf
sf.write("output.wav", audio, samplerate=22050)
```


これでフォルダ内に合成音声のWAVファイルが保存されます。日本語・英語・中国語の各入力について異なるファイル名で保存し、正しく音声再生できるか確認してください。

5. **中間出力（メルスペクトログラム）の可視化:** 最後に、中間表現であるメルスペクトログラムを画像として保存します。Matplotlib等を使ってスペクトログラムをプロットできます。以下は簡単なプロット例です。

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10,4))
plt.imshow(mel, aspect='auto', origin='lower',
            interpolation='nearest', cmap='viridis')
plt.colorbar(label="Amplitude (dB)")
plt.title("Mel Spectrogram")
plt.xlabel("Frame Index")
plt.ylabel("Mel Bin")
plt.tight_layout()
plt.savefig("mel_spectrogram.png")
```

上記により、`mel_spectrogram.png` という画像ファイルにメルスペクトログラムが可視化されて保存されます。人間の音感に近い対数メル尺度で表示され、縦軸が周波数方向（メルバンド）、横軸が時間フレームを表します。音声合成の成否を視覚的に確認する手段として、有用な中間出力です。

動作確認と次ステップ

以上の手順により、多言語対応のVITSベースTTSパイプラインの**初期検証**が完了します。日本語・英語・中国語それぞれについて、音声波形（.wav）と対応するメルスペクトログラム画像が得られているはずです。最後に以下の点を確認しましょう。

- 合成音声を実際に再生し、入力テキストの内容がおおむね発話されているか（日本語・英語・中国語で適切な音声になっているか）をチェックします。
- メルスペクトログラムを目視で確認し、音声らしいスペクトルパターン（フォルマントの横縞やピッチに対応する斜めの縞など）が現れているかを見ます。特に発話に失敗している場合、スペクトログラムが不自然（ランダムノイズ状、あるいは真っ黒/真っ白など）になることがあります。

もし発話品質に問題がある場合、原因切り分けとして以下を検討してください:

- 音素変換やXPhoneBERT統合に起因する問題かを調べるため、一度XPhoneBERTを使わず**通常のVITS推論**を各言語で行い比較する。例えば英語ならテキストを直接既存の英語VITSモデルに与えて音声を出力し、BERT統合版と聴き比べます。品質差が大きい場合、統合部分の調整が必要です。
- XPhoneBERT統合後のモデルを**追加微調整**してみる。少量でも良いので対象言語の音声データでファインチューニングすることで、BERT埋め込みからメルへの変換が安定し、品質向上が見込めます。
- 各言語で音素記号の扱いが適切か確認する。例えば日本語の音素がモデルの訓練時と表記揺れがあれば、対応する辞書の修正や変換ルールの追加が必要です。

以上、段階的に実装と検証を行うことで、多言語対応のVITSベースTTSシステムのプロトタイプを構築できます。最終的には、XPhoneBERTの多言語表現力とVITS+BigVGANの高品質な音声合成力を組み合わせることで、各言語で自然な音声合成が可能になることが期待されます ¹⁰ ²⁴。お疲れさまでした！

参考文献・ソース: 本手順では、XPhoneBERT [3](#) [8](#)、VITS [20](#)、BigVGAN [27](#) などの公式リソースおよび実装例を参照しています。各モデルの詳細やライセンスについてはそれぞれのGitHub/論文をご確認ください。

[1](#) [2](#) [3](#) [4](#) [6](#) [7](#) [8](#) [9](#) [10](#) [11](#) [GitHub - VinAIRResearch/XPhoneBERT: XPhoneBERT: A Pre-trained Multilingual Model for Phoneme Representations for Text-to-Speech \(INTERSPEECH 2023\)](#)

<https://github.com/VinAIRResearch/XPhoneBERT>

[5](#) [23](#) [GitHub - HuuHuy227/XphoneBert_Vits2: VITS2 extended with XPhoneBERT encoder](#)

https://github.com/HuuHuy227/XphoneBert_Vits2

[12](#) [13](#) [facebook/mms-tts · Hugging Face](#)

<https://huggingface.co/facebook/mms-tts>

[14](#) [15](#) [Why there is no Chinese in MMS? · Issue #5145 - GitHub](#)

<https://github.com/facebookresearch/fairseq/issues/5145>

[16](#) [GitHub - Edresson/YourTTS: YourTTS: Towards Zero-Shot Multi-Speaker TTS and Zero-Shot Voice Conversion for everyone](#)

<https://github.com/Edresson/YourTTS>

[17](#) [18](#) [20](#) [Pre-trained models — sherpa 1.3 documentation](#)

https://k2-fsa.github.io/sherpa/onnx/tts/pretrained_models/index.html

[19](#) [litagin/vits-japros-pretrained · Hugging Face](#)

<https://huggingface.co/litagin/vits-japros-pretrained>

[21](#) [30](#) [31](#) [32](#) [VITS](#)

https://huggingface.co/docs/transformers/en/model_doc/vits

[22](#) [jackyqs/vits-aishell3-175-chinese at main](#)

<https://huggingface.co/jackyqs/vits-aishell3-175-chinese/tree/main>

[24](#) [27](#) [28](#) [29](#) [nvidia/bigvgan_24khz_100band · Hugging Face](#)

https://huggingface.co/nvidia/bigvgan_24khz_100band

[25](#) [26](#) [GitHub - NVIDIA/BigVGAN: Official PyTorch implementation of BigVGAN \(ICLR 2023\)](#)

<https://github.com/NVIDIA/BigVGAN>